

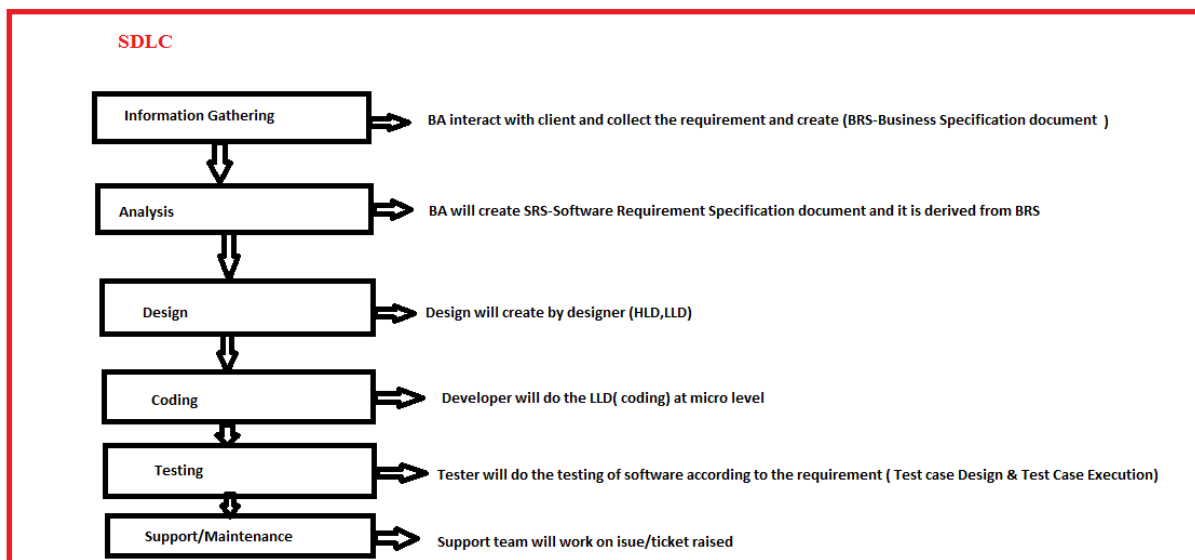
UNIT 2

SDLC (Software Development Life Cycle) →

Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC is a methodology that defines the entire procedure of software development step-by-step.

Software Development Life Cycle involves different stages:

1. Information Gathering
2. Analysis
3. Design
4. Coding
5. Testing
6. Support/Maintenance



Before going forward let's see the project team to learn the SDLC life cycle:

1. Delivery Manager → proper delivery of the requirement to the client
 2. Project Manager → Handle the team (work to be done)
 3. Business Analyst → Collect the requirement from the client
 4. Designer/Solution Designer → Architect the software
 5. Developer team → to write the code/ to develop the software
 6. Tester (QA) team → Test the software/build/application so that software should be defect free
- This is how the team works in an organization.

- **Information Gathering (BRS Document)→**

1. Business Analyst (BA) will collect the requirement from client.
2. Define the BRS document (Business Requirement Specification)
3. BRS→total business requirement → Platform is going to developed according requirement
4. Ex-Zeroda, Upstock etc. → End user will use this application use for share market activity
Buy share- 1000/-→sell share-1200/-→Profit=200/- → Brokerage charge-100/-
Customer final profit =100/-

- **Analysis (SRS Document)→**

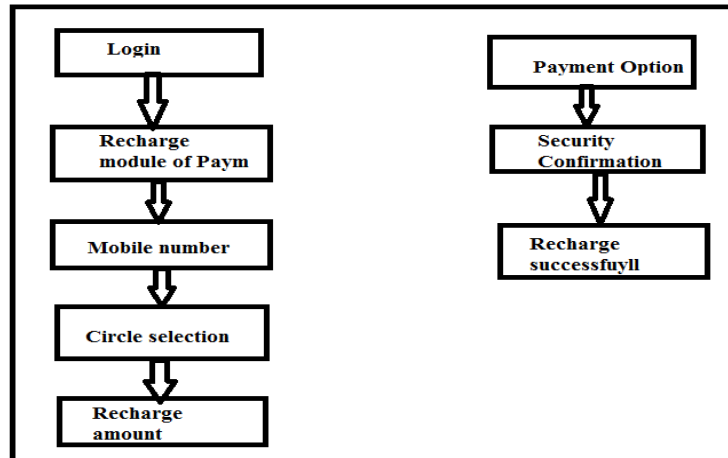
SRS→ Software Requirement Specification/Functional Requirement Specification/Customer Requirement Specification

SRS will derive from BRS →this document will prepare by BA

SRS→defined functional requirement to be developed & system requirement

SRS will contain→

1. **Functional Requirement**→Attribute which are required to be developed Ex-Login ID, password, submit button for Paytm application
2. **Functional Flow Diagram**→Flow of task which we (customer) are going to perform Ex- Paytm→login with id & password→Mobile recharge→mobile no (10 digit)→network select(idea,Airtel)→Region(MH)→recharge(amount)→transaction's(UPI,net banking/Debit card/credit card)→recharge successful message.

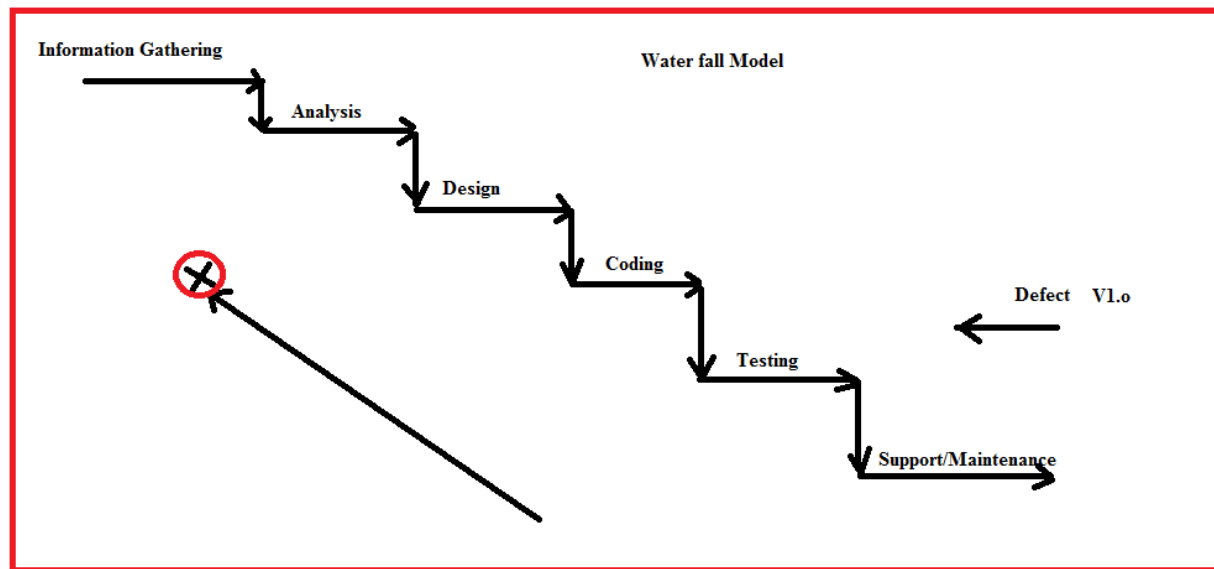


3. **Use cases**→Single requirement of project→ **A)Description**→Details requirement about the particular use case Ex-Recharge module of paytm **B)Acceptance Criteria**→ Does and donts about the requirement Ex- Paytm →For login →mobile no should be 10 digit, mobile no should not be less than 10 digit

4. **Screen shot/snapshot/prototype**→ BA create the prototype model for visualization of functionality before the software development (BA-use Iris software for prototype)
5. **Design**→Design prepared by designer, designer will prepare HLD&LLD
Ex-Recharge module in Paytm→HLD→
 - 1.Mobile no text box & password –textbox
 - 2.circle selection
 - 3.plan selection
 - 4.payment page
 5. Security confirmation
 - 6.Success message/thanks message
6. **Coding**→In coding stages developer is working on LLD
7. Ex-LLD→Paytm→Rechargemodule→LoginID→Password(strong-Ex-Abcd@123)
→submit button→paytm page open→Icon needs to show→recharge module→new popup→recharge activities (End to end functionality development→LLD)
8. **Testing**→ Testing is performed by tester team when we are starting the testing→ after the development completed and build/software deploy/send in our SIT environment. As tester test case design, test case execution.
Ex- Email→ Hi abc (Tester), we have completed development end to end you can start with testing in SIT environment. URL-111.123/paytm/recharge.
9. **Maintenance/Support**→ Support team works on issues/current existing issues Ex-Paytm→Recharge→Recharge fail→transaction is successful and money also deducted→ on that issue support team will works over the ticket raised/issue.

Water fall Model →

- Waterfall model/process is defined as sequential model of software development.
- After completion of one stage, we will move towards the next stage.
- Ex-After the completion of information gathering will moves towards the Analysis.
- We can't move toward the previous stage after the completion of this stage means we can't move back Ex- After the completion of design while coding if some part is missing even though we can't move to design
- Duration of development and delivery of software more than 3 months.
- If we get the defect while testing then also it is not possible to resolve it in present version so it will resolve in next version of software Ex- If we get defect in V1.0 then defect will resolve in version V1.1.
- If we will get defect then it will store in sheet and will send to the BA and while next version development it will resolve.
- Ex-Video games application.



Drawback of waterfall model →

1. We can't go to previous stage
2. Delivery /deployment of the software time duration is very large or not fix (Fix time line not possible)
3. We can't accept new change request from client

Spiral Model →

- The spiral model works in an iterative nature. It is a combination of both the Prototype development process and the Linear development process (waterfall model).
- This model places more emphasis on risk analysis. Mostly this model adapts to large and complicated projects where risk is high.
- Every Iteration starts with planning and ends with the product evaluation by the client.

Spiral model undergoes 4 phases:

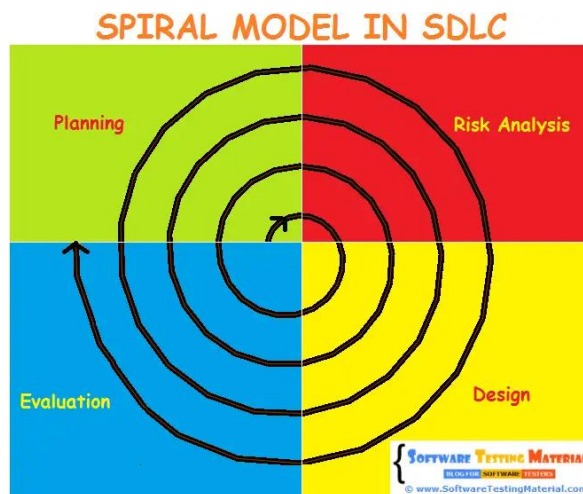
- Planning Phase – Requirement Gathering, Cost Estimation, Resource Allocation.
- Risk Analysis Phase – Strengths and weaknesses of the project.
- Design Phase – Coding, Internal Testing and deployment.
- Evaluation Phase – Client Evaluation (Client-side Testing) to get the feedback.

- Advantages:

- It allows requirement Changes.
- Suitable for large and complicated projects.
- It allows better risk analysis.
- Cost effective due to good risk management.

- Disadvantages:

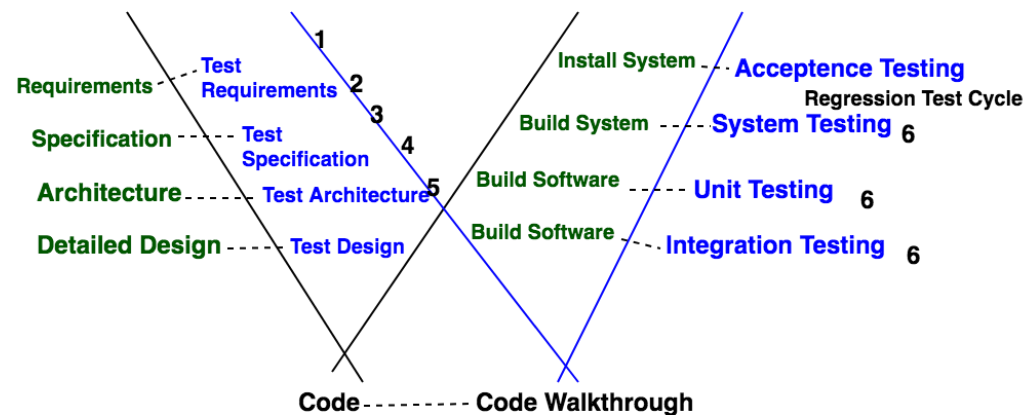
- Not suitable for small projects.
- The success of project depends on risk analysis phase.
- Have to hire more experience resource especially for risk analysis.



W Model →

- V-model and W-model are two of the most important models that are used in software testing.
- W-Model covers those activities that are skipped by V-Model and also, It deals with problems that couldn't be caught by V-Model.
- W-Model approach attempts to address and tackle the shortcomings of V-Model.

- W-model can be done only once the development of the product is complete with no modifications required to be done in between. This type of testing is most suitable for short projects.
- With the help of W-Model, we ensure that the testing of the product starts from the very first day of the inception of the product, and each phase of the product development is verified and validated.



Phases of W-Model:

Each phase is verified/validated. Dotted line shows that every phase in green is validated/tested through every phase in sky blue. Now, in the above figure,

1. Point 1 refers to – Build Test Plan & Test Strategy.
2. Point 2 refers to – Scenario Identification.
3. Point 3 refers to – Test case preparation from Specification document and design documents.
4. Point 4 refers to – Test case preparation from Specification document and design documents.
5. Point 5 refers to – review of test cases and update as per the review comments.
6. Point 6 refers to – Various testing methodologies such as Unit/integration testing, path testing, equivalence partition, boundary value, specification-based testing, security testing, usability testing, performance testing.
7. After this, there are regression test cycles and then User acceptance testing.

Advantages of W-Model:

- In W-Model there is no strict division between constructive tasks on the left-hand side and the more destructive tasks on the right-hand side.
- During the test phase, the developer is responsible for the removal of defects and the correction of the implementation.
- Emphasis the fact that testing is more than just construction, execution and evaluation of test cases.
- The importance of the tests and the ordering of the individual activities for testing are clear.

Disadvantages Of W-Model:

- The real facts are simplified in this model.
- There is a need for a simple model if all people involved in a project are to accept it.
- For highly critical applications the test activities certainly have higher weighting or at least equal weighting with other activities.

V-Model (Process)→

- V- stands for verification & validation.
- V- model defines, it is mapping between development stage and testing stage.
- V-model is nothing but plan driven process.
- In a V-model delivery/deployment of software/build/application duration within 3 months.
- In a V-model-parallel development & testing is going on.
- V-model deals with parallel life cycle development and life cycle testing.

The V-Model is a software development life cycle (SDLC) model that provides a systematic and visual representation of the software development process. It is based on the idea of a “V” shape, with the two legs of the “V” representing the progression of the software development process from requirements gathering and analysis to design, implementation, testing, and maintenance.

LCD (Life cycle development)

Information gathering (BRS)

Analysis (SRS/FRS)

LCT (Life cycle Testing)

Assessment of development plane

Prepare **test plane**

Requirement Testing/ Understand

Design (**HLD**, LLD)

Design phase testing (**review**)

Coding (**dev team-LLD**)

Program phase testing (**WBT**) → unit & Integration

Test case design (**TCD**)

Build/ Application (Deployment)

System & Function testing (**BBT**) – (**TCE**) → office

User acceptance testing (**UAT**) → US (Client side)

Knowledge transfer (**KT**) → Client

Maintenance/ **Support (Tickets)**

CR (Change request) /

DRE (defect removal efficiency)

TCD → Test Case Design, TCE → Test case Execution means testing (all functional and nonfunctional testing)

UAT → User Acceptance Testing → which is done at client location like-US/UK.

CR → Change request Ex- We have developed Paytm Recharge module for mobile → Client is asking for add the +91 before the mobile number → support team will work over.

Ex- Large change request → Thanks message → client is asking for generate invoice link should be send over customer mobile number so that customer will see details of transactions and recharge details → works the Project team.

DEFECT REMOVAL EFFICIENCY (DRE) = $A/(A+B)$

A = Defect found in BBT (Black Box Testing)

B = Defect found in UAT

DRE = $1/(1+0) = 1$ (ideal)

Advantages of V-Model:

- This is a highly disciplined model and Phases are completed one at a time.
- V-Model is used for small projects where project requirements are clear.
- Simple and easy to understand and use.
- It enables project management to track progress accurately.

Disadvantages of V-Model:

- High risk and uncertainty.
- It is not good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contain a high risk of changing.

- Time-Consuming: The V-Model can be time-consuming, as it requires a lot of documentation and testing.

Agile Model (Process) / Agile Methodology →

- Agile Model (process)→ It is a continuous development and iterative process for the software development.
 - Agile process is the value driven (We as software engineer giving very high priority to the client).
 - It is mostly used methodology used in an organization (90-95% organization used).
 - If any change request (CR) comes from client then our team will be check either this change is impact on current development or not. If not then we will accept the change request and implement in current development. If change request is impacted on current development, then we will not accept this CR in current sprint so we will consider in next sprint.
 - Duration of delivery/deployment of the software in Agile is → 2 week/3 week/ 1 month, depends on company policy.
 - Agile will have sub types (Frame works)→
 1. Scrum Agile →(Sprint wise delivery/deployment of the software)
 2. Kanban
 3. Lean
 4. XP (Xtream Programming)
 5. FDD (Future Driven Development)
- } Support team
- So, we are working in Scrum Agile Process → It is 2 weeks/3-week process.

Let's see the difference between the SDLC and Agile so we got a clear idea about agile methodology.

SDLC	Agile Methodology
Information Gathering (BRS)	Product Backlog → n number of requirement collected from stake holder (Client) and prepared by the product owner (BA)
Analysis (SRS) → Functional Requirement, Function flow diagram, Use case→single req, Description→Details about the requirement Acceptance Criteria→Does & don't about requirement	Sprint Backlog → In particular sprint duration on how much requirement we are covering (small parts of product backlog is nothing but sprint backlog) Decides→according to Stake holder (client) priority

Prototype/snapshot/screen shot→Visualization of the software	User story → single requirement Description→ Details about requirement Acceptance Criteria→ Does & don't about requirement (SRS document→ functional details about requirement)
Design→HLD, LLD	Design→HLD, LLD
Coding→LLD	Coding→LLD
Testing→TCD, TCE	Testing→TCD, TCE
Support/Maintenance→works on Ticket/issues	Support/Maintenance→works on Ticket/issues

Product Backlog→It is a total number of requirements of the project and it contains all the requirement of all the modules Ex-Paytm project requiremts→Ex-Recharge module, DTH recharge module, Electricity bill, LIC premium payment etc.

Sprint Backlog→ Product backlog is divides into small parts and single small part(requirement) is nothing but print backlog. In that user story→single requirement Ex-Only recharge submodule of Paytm

Description→Details about the requirement and Acceptance Criteria→ Does & donts about the requirements

Terminologies used in agile to the different team members.

Delivery Manager	Solution Master
Client	Stake holder
Business Analyst (BA)	Product Owner
Project Manager	Scrum Master

Agile Testing Methodologies:

Some of the agile testing methodologies are:

1. Test-Driven Development (TDD):

- TDD is the software development process relying on creating unit test cases before developing the actual code of the software. It is an iterative approach that combines 3 operations, programming, creation of unit tests, and refactoring.

2. Behavior Driven Development (BDD):

- BDD is agile software testing that aims to document and develop the application around the user behavior a user expects to experience when interacting with the application. It encourages collaboration among the developer, quality experts, and customer representatives.

3. Exploratory Testing:

- In exploratory testing, the tester has the freedom to explore the code and create effective and efficient software. It helps to discover the unknown risks and explore each aspect of the software functionality.

4. Acceptance Test-Driven Development (ATDD):

- ATDD is a collaborative process where customer representatives, developers, and testers come together to discuss the requirements, and potential pitfalls and thus reduce the chance of errors before coding begins.

5. Extreme Programming (XP):

- Extreme programming is a customer-oriented methodology that helps to deliver a good quality product that meets customer expectations and requirements.

6. Session-Based Testing:

- It is a structured and time-based approach that involves the progress of exploratory testing in multiple sessions. This involves uninterrupted testing sessions that are time-boxed with a duration varying from 45 to 90 minutes. During the session, the tester creates a document called a charter document that includes various information about their testing.

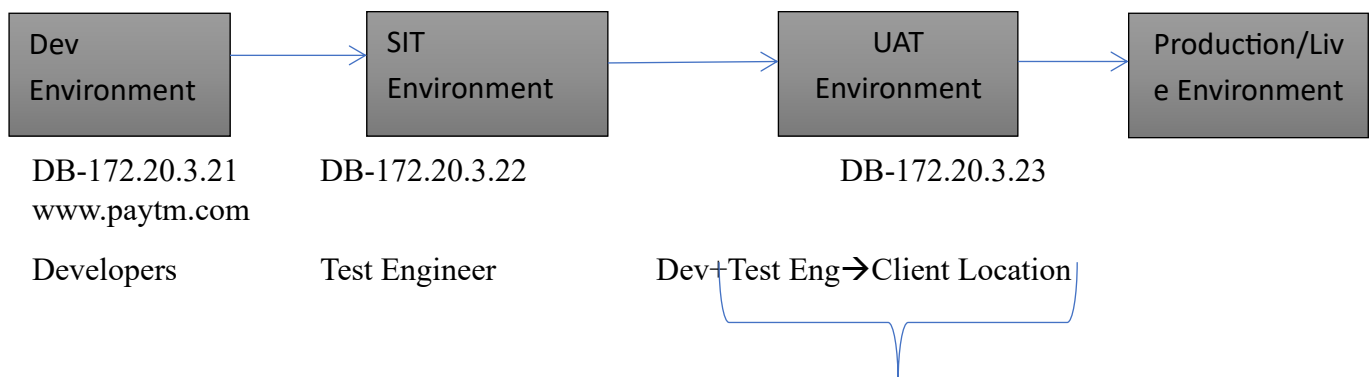
7. Dynamic Software Development Method (DSDM):

- DSDM is an agile project delivery framework that provides a framework for building and maintaining systems. It can be used by users, developers, and testers.

8. Crystal Methodologies:

- This methodology focuses on people and their interactions when working on the project instead of processes and tools. The suitability of the crystal method depends on three dimensions, team size, criticality, and priority of the project.

Environments in Project →



	(Pune)	(US or any client location)
Testing→	WBT	Blackbox Testing
		UAT Testing→
1.Unit Testing	1.Smoke/Sanity	1. Alpha Testing
2.Integration Testing	2. Functional/non function	2. Beta Testing
	3.Regression	

Agile Meeting→

In an organization mostly they are following 5 types of Agile meetings→

- 1.Grooming Meeting
- 2.Sprint planning Meeting
- 3.Daily standup meeting/Scrum meeting
- 4.Sprint Review meeting
- 5.Sprint Retrospective meeting

Meeting Type	Purpose	Include
1.Grooming Meeting (Before start of the sprint)	To clear the doubts about the user story (requirements)/SRS document (If US/UK project→Meeting time→ 7 PM IST)	Product Owner (BA)→Chairperson, Designer, developer team, tester team, scrum master→Optional (meeting→30 minutes)

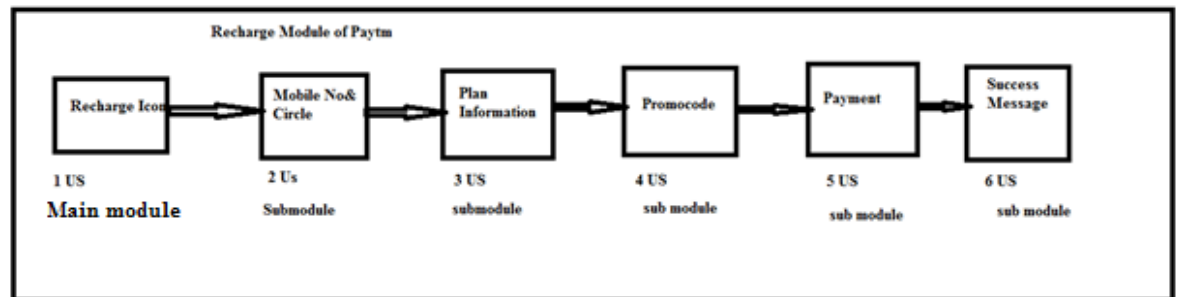
2.Sprint Planning Meeting (1 st Day of the sprint)	In current sprint how much user stories we are deploying and need to give estimation (time span) Ex-in current sprint we will take 5 user story →how much time will be required for development & testing will be discussed	Scrum Master→chair person, Designer, Developer team, tester team, Product owner (Meeting→30 minutes)
3.Daily Standup/scrum meeting (Every Day)	Daily Updates →1.What we have done yesterday 2.What we are doing today 3.Issues/roadblocks→difficulties in working	Scrum Master→Chair person Designer, Developer team, tester team, product owner (BA) (meeting→15 min) (Timing →10 AM IST)
4.Sprint Review Meeting (Last day of the sprint)	Functionality developed will test and giving the demo of functionality which is developed in current sprint to the stake holder (Client).	Meeting → chairperson→BA (Product owner), Involved→Client (stake holder), designer, developer team, tester team, Scrum master (meeting→ 1 hrs. duration)
5.Sprint Retrospective Meeting (Last day of the sprint)	1.Whatever we have done good things in current sprint 2. Whatever we have done wrong things in current sprint That things will be discussed in this meeting and whatever mistakes/wrong thing happened in current sprint that should not be happen in next sprint.	Chairperson→Scrum master, Involved→Developer team, tester team, product owner→optional

Levels Of Testing:

1. Unit Testing:

- Unit Testing is the first level of software testing, which is used to test if software modules are satisfying given requirement or not.
- The first level of testing involves analyzing each unit or an individual component of the software application.
- Unit testing is also the first level of functional testing. The primary purpose of executing unit testing is to validate unit components with their performance.

- In Unit testing Developer will do the unit testing after the completion of coding from the user story
- Developer will test the sub module of the application this nothing but unit testing
- Ex-Recharge module of Paytm



2. Integration Testing:

- The second level of software testing is the integration testing. The integration testing process comes after unit testing.
- Integration testing is performed by developer
- Integration testing is nothing but combination of all the sub modules testing
- There are 2 types of Integration Testing → 1. Front End Integration Testing 2. Backend Integration Testing
- To perform the integration testing → Developer will apply different approaches →
 - **Top-down Approach** → In top-down approach if sub module is under development or incomplete then developer will create one temporary file is called stub file in xml format and developer will do the integration testing.
 - **Bottom-up Approach** → In the bottom-up approach main module or starting module is not completed or under development then developer will create one temporary file in xml format and it is called as Driver file and developer will complete the integration testing.
 - **Sandwich Approach** →
 - If main/starting module is under development or incomplete and end sub module also incomplete or under development, middle modules are complete then developer will use sandwich approach
 - In this case developer will create one temporary file in xml for starting module is called driver file & for end sub module developer will create one temporary xml file called stub file and complete the integration testing

System Testing:

- The third level of software testing is system testing, which is used to test the software's functional and non-functional requirements.
- It is end-to-end testing where the testing environment is parallel to the production environment. In the third level of software testing, we will test the application as a whole system.
- To check the end-to-end flow of an application or the software as a user is known as System testing.
- In system testing, we will go through all the necessary modules of an application and test if the end features or the end business works fine, and test the product as a complete system.

Acceptance Testing:

- The last and fourth level of software testing is acceptance testing, which is used to evaluate whether a specification or the requirements are met as per its delivery.
- The software has passed through three testing levels (Unit Testing, Integration Testing, System Testing). Some minor errors can still be identified when the end-user uses the system in the actual scenario.
- Acceptance testing is the squeezing of all the testing processes that are previously done.
- The acceptance testing is also known as User acceptance testing (UAT) and is done by the customer before accepting the final product.
- User Acceptance Testing is done at client location like-US/UK.

Static Testing:

- Static Testing also known as Verification testing or non-execution testing is a type of Software Testing method that is performed in the early stage of development to avoid errors as it is easier to find sources of failures and it can be fixed easily.
- Static can be done manually or with the help of tools to find bugs and improve the quality of the software.
- It helps to find errors in the early stage of development which is also called the verification process.
- It enhances maintainability and ultimately saves time and money in the long run.

- Static testing is needed whenever the following situations are encountered while testing an application or software:
- **Increased software size:** Static testing is required to get free from bugs in the early stages of development as with testing activity the size of the software increases which is difficult to handle due to a reduction in the productivity of the code coverage.
- **Dynamic testing is expensive:** Dynamic testing is more expensive than static testing as dynamic testing uses test cases that have been created in the initial stages and there is also a need to preserve the implementation and validation of the test cases which takes a lot of time from test engineers:
- **Dynamic testing is time-consuming:** Static testing is required as dynamic testing is a time-consuming process.
- **Bugs detection at early stages:** Static testing is helpful as it finds bugs at early stages, while dynamic testing finds bugs at later stages which makes it time-consuming and costly to fix the bugs.
- **Improvement of development productivity:** Static testing helps to identify bugs early in the software development thus it helps to reduce the flaws during production and increase development productivity.